

MASTERING PROMETHEUS

Prometheus가 관측값을 믿을 만한 신호로 만드는 과정

데이터가 태어나 저장되고, 대상을 찾고, 경고가 사람에게 도착하기까지를 한 흐름으로 봅니다.

오늘 따라갈 한 줄

대상을 찾는다 → 값을 굵는다 → 시계열로 저장한다 → 규칙을 평가한다 → 알림을 묶어 사람에게 보낸다.

다섯 단계를 순서 그대로 따라갑니다.

먼저 짚을 용어 넷

- **메트릭**: 측정 대상의 이름
- **라벨**: 같은 메트릭을 쪼개는 꼬리표
- **스크레이프**: 서버가 `/metrics` 를 주기적으로 가져가는 일
- **알림**: 조건이 일정 시간 참일 때 생기는 상태

오늘의 순서

1. **수집** — 무엇을 굵을지 고르고, 값은 누가 만드는가
2. **저장** — 그 값이 어떤 모양으로 쌓이는가
3. **조회** — 쌓인 값을 어떻게 읽는가
4. **알림** — 두 프로그램의 책임 분리

맨 위 문장의 다섯 단계를 순서 그대로 따라갑니다. 값이 태어나는 자리에서 시작해, 사람에게 도착하는 데서 끝납니다.

STEP 1 / 4

수집 — 무엇을 긁고, 값은 누가 만드는가

대상을 찾는다 → 값을 긁는다

서비스 디스커버리

relabeling

Exporter · collector

디버깅

서비스 디스커버리는 후보만 만듭니다

Prometheus는 스스로 대상을 알지 못합니다. "어디를 긁어야 하는지" 주소 목록을 대신 받아 오는 일이 서비스 디스커버리입니다.

파드는 뜨고 지고 IP도 계속 바뀝니다. 그래서 주소를 설정 파일에 적지 않습니다. 대신 **API**에 주기적으로 다시 물어봅니다.

API에 묻는다 → **후보 목록** → relabeling이 고른다 → 남은 주소를 긁는다

발견

후보 목록과 `__meta_*` 라벨을 받습니다.

아직 아무것도 긁지 않은 상태입니다.

선별

relabeling이 남길 대상을 결정합니다.

여기서 후보 수가 줄어듭니다.

스크레이프

살아남은 최종 주소로 요청합니다.

이때 비로소 값이 생깁니다.

SD는 후보만 냅니다. 값은 exporter가 만들고, 무엇을 긁을지는 relabeling이 정합니다 — 셋의 책임이 다릅니다.

relabeling은 두 번 돌고, 거르는 것이 다릅니다

이름이 닮아 헷갈리지만 **놓인** 자리가 다르고, 그래서 거르는 대상도 다릅니다.

앞의 것은 **긋기 전에** 돌아 대상을 고르고, 뒤의 것은 **저장하기 전에** 돌아 시계열을 고릅니다.

	relabel_configs	metric_relabel_configs
언제 도는가	스크레이프 전	저장 전
무엇을 거르는가	타겟 — 긋을 대상	시계열 — 저장할 값
무엇이 줄어드는가	스크레이프 부하 · 네트워크	카디널리티 · 디스크
못 하는 일	이미 받은 값은 못 거름	긋는 것 자체는 못 막음

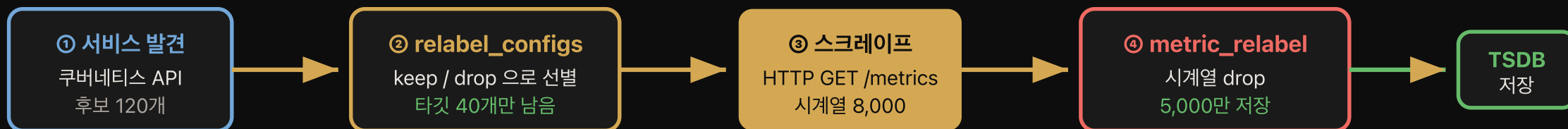
특정 파드를 *아예* 긋지 않으려면 어느 쪽일까요?

정답: **relabel_configs** 입니다. metric 쪽은 이미 긋은 뒤라 부하가 그대로 듭니다.

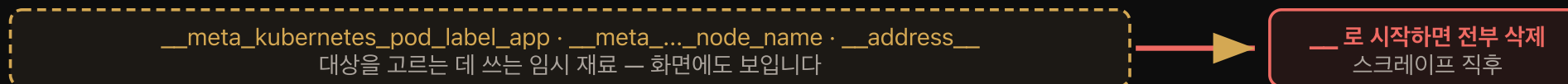
후보에서 저장까지, 무엇이 걸러지는가

단계마다 걸러지는 것도, 남는 라벨도 달라집니다.

각 단계에서 무엇이 몇 개 남는지를 함께 보면 두 relabeling의 차이가 드러납니다.



라벨이 살아 있는 구간



남기려면 옮겨 담아야 합니다

source_labels: [__meta_kubernetes_pod_node_name] → target_label: node (이제 저장됩니다)

주소를 받는다

값을 대상을 고른다

값을 가져온다

저장할 값을 고른다

여기까지는 타겟 개수가 줄어듭니다

여기서는 시계열 개수가 줄어듭니다

규칙 하나는 부품 다섯으로 읽습니다

규칙은 "어떤 라벨을 보고, 무엇에 맞으면, 어떻게 한다"의 반복입니다.

아래는 **app=api** 라벨이 붙은 파드만 굵겠다는 규칙입니다.

```
relabel_configs:
- source_labels:
  [__meta_..._label_app]
  regex: api
  action: keep
```

이 네 줄은 이렇게 읽습니다.

1. 파드에 붙은 **app** 라벨 값을 본다
2. 그 값이 **api** 와 정확히 같은가
3. 같으면 남기고, 아니면 버린다

필드	뜻
<code>source_labels</code>	어떤 라벨의 값을 볼 것인가
<code>regex</code>	그 값이 이 패턴에 맞는가 (기본 <code>(.*)</code>)
<code>action</code>	맞으면 무엇을 할 것인가 (기본 <code>replace</code>)
<code>target_label</code>	어느 라벨에 쓸 것인가
<code>replacement</code>	실제로 쓸 값 (기본 <code>\$1</code> 첫 캡처)

`regex` 는 양쪽 끝이 앵커링됩니다. 부분 일치를 원하면 `.*api.*` 로 풀어야 합니다.

후보 수를 바꾸는 액션은 둘뿐입니다

`action` 이 대상을 줄일지, 라벨만 손댈지를 가릅니다.

현행 문서는 액션을 11개 정의하지만, 실무에서 손이 가는 것은 아래 다섯입니다.

```
action: keep      # 남길 것만
action: drop      # 뺄 것만

action: replace   # 값 옮기기
action: labeldrop
action: labelmap
```

`keep` 과 `drop` 은 서로 반대라 하나로 같은 일을 표현할 수 있습니다. 정규식이 단순해지
는 쪽을 고르는 것이 관례입니다.

액션	하는 일	후보 수
<code>keep</code>	매칭되는 것만 남김	줄어들
<code>drop</code>	매칭되는 것을 버림	줄어들
<code>replace</code>	읽은 값을 옮겨 씀	안 바뀜
<code>labeldrop</code>	라벨 이름을 지움	안 바뀜
<code>labelmap</code>	라벨 이름을 복사	안 바뀜

`replace` 는 값을, `labeldrop` 은 라벨 자체를 다룹니다. 이름이 비슷해 자주 헷갈리는 자리입니다.

쿠버네티스에서는 Operator가 대신 씁니다

지금까지 본 SD 설정과 relabeling 규칙을 손으로 쓰지 않습니다.

사람은 **ServiceMonitor**에 "어느 서비스의 어느 포트를 긁어라"만 적습니다. 그것을 Operator가 실제 **scrape** 설정으로 번역합니다.

ServiceMonitor 작성 → Operator가 번역 → **prometheus.yaml** 생성 → 사이드카가 재적재

긁을 대상	사람이 쓰는 것	Operator가 하는 일
K8s Service·Pod	<code>ServiceMonitor</code> · <code>PodMonitor</code>	selector를 <code>kubernetes_sd</code> + relabel 규칙으로 번역
블랙박스 프로빙	<code>Probe</code>	대응 SD와 스크레이프 설정 생성
EC2 · 사내 SD	<code>additionalScrapeConfigs</code>	번역 없음 — 처음부터 SD 문법으로 직접

두 경로는 번역 단계가 있느냐로 갈립니다. `additionalScrapeConfigs` 는 YAML을 직접 받지 않고 **Secret**의 키를 가리킵니다 — 스크레이프 설정에 자격증명이 섞이기 쉬워 평문 필드를 피한 설계입니다.

안 굶힐 때는 증상으로 화면을 고릅니다

여기까지가 대상을 고르는 과정입니다. 문제는 대개 두 화면 중 하나에서 갑니다.

`/service-discovery` 는 발견과 선별을, `/targets` 는 연결을 보여 줍니다. 앞이 정상인데 뒤가 DOWN이면 고칠 곳은 대상 쪽입니다.

증상	볼 화면	lastError	볼 곳
타깃이 아예 없음	<code>/service-discovery</code>	connection refused	대상의 바인딩 주소
라벨이 예상과 다름	<code>/service-discovery</code>	404 Not Found	<code>metrics_path</code>
타깃은 있는데 DOWN	<code>/targets</code>	context deadline exceeded	<code>scrape_timeout</code>

`up=0` 은 실패했다는 사실만 말합니다. kubeadm 클러스터의 `connection refused` 는 대개 메트릭이 루프백에만 바인딩된 것이지 설정 실수가 아닙니다.

여기까지는 전부 주소를 고르는 일이었습니다. 그런데 주소를 찾아가도 거기에 값이 저절로 있는 것은 아닙니다. 그 `/metrics` 를 열어 두고 값을 내놓는 쪽이 따로 있습니다.

Exporter는 다른 시스템의 값을 번역합니다

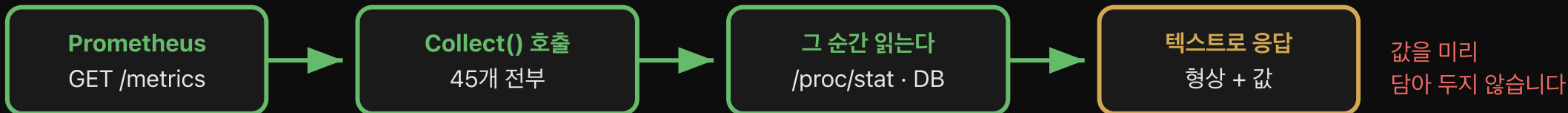
주소를 고르는 일과 값을 만드는 일은 다른 쪽이 맡습니다. 값을 내놓는 쪽이 exporter입니다.

서비스 디스커버리가 "어디로 갈지"를 정했다면, exporter는 그 자리에서 "무엇을 줄지"를 정합니다. 부품은 셋이고 각각 다른 시점에 일합니다.

① 등록 시점 — 한번



② 스크레이프 시점 — 매번

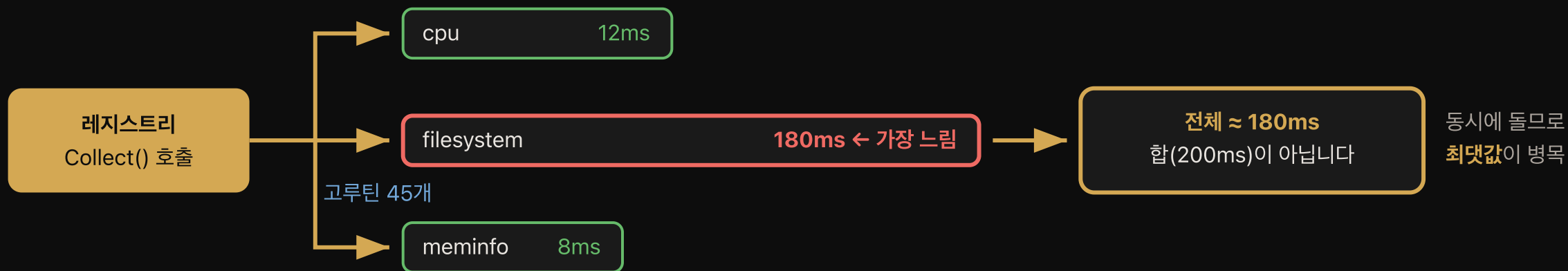


Describe 는 등록 때 형상만, Collect 는 스크레이프 때 그 순간의 값을 냅니다. 그래서 스크레이프가 방아쇠입니다.

collector는 각각 고루틴으로 동시에 돕니다

Node Exporter는 collector마다 **별도 고루틴**을 띄웁니다. 그래서 개수가 늘어도 느려지지 않습니다.

전체 응답 시간은 **합이 아니라 가장 느린 하나**입니다.



속도를 높이려고 collector 수를 줄일 이유는 없습니다. 다만 이는 **Node Exporter**에 국한된 이야기입니다.

메모리는 free와 available을 구분합니다

둘을 헷갈리면 **멀쩡한 서버가 메모리 부족으로 보입니다.**

리눅스는 남은 메모리를 캐시로 쓰기 때문에 **free**는 원래 작습니다.

메트릭	뜻
<code>node_memory_MemTotal_bytes</code>	설치된 총 메모리
<code>node_memory_MemAvailable_bytes</code>	쓸 수 있는 메모리 — free + 회수 가능분
<code>node_memory_MemFree_bytes</code>	즉시 쓸 수 있는 free만

대시보드와 임계값에는 `MemAvailable` 을 씁니다. `MemFree` 는 캐시로 쓰이는 메모리를 **부족한 것으로 잘못 셉니다.**

스크레이프가 느리면 **duration**부터 봅니다

앞 장의 "**가장 느린 하나**"를 실제로 찾아내는 방법입니다.

collector별 소요 시간이 메트릭으로 나옵니다.

```
node_scrape_collector_duration_seconds
```

먼저 볼 것

`scrape_timeout`에 가까운 collector 하나를 찾습니다. 느리면 `--no-collector.<이름>`으로 끕니다.

오해하기 쉬운 것

`node_scrape_collector_success=0`은 실패가 아니라 **하드웨어 부재**일 수 있습니다. 그 장비에 없는 항목이면 정상입니다.

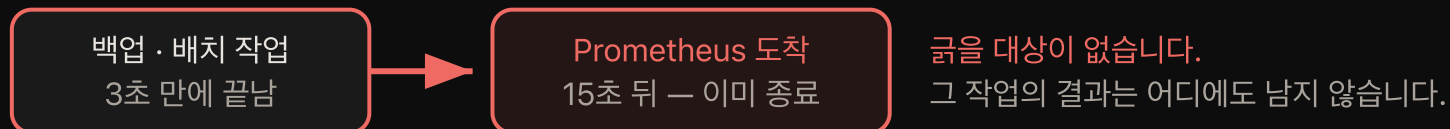
전부 켜 두는 것이 항상 옳지는 않습니다. **쓰지 않는 collector는 끄는 것**이 스크레이프 시간을 줄이는 가장 단순한 방법입니다.

textfile collector는 짧은 작업의 기록을 노출합니다

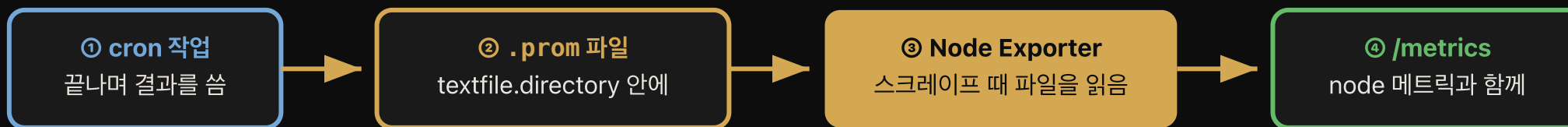
굵으러 왔을 때 이미 끝나 있는 작업의 결과를 남기는 방법입니다.

파일을 중간 보관소로 씁니다. 작업은 쓰고, Node Exporter는 읽습니다.

문제 — 짧은 작업은 굵힐 때 이미 없습니다



해법 — 파일에 적어 두고 대신 읽게 합니다



작업이 죽어 파일이 갱신되지 않아도 **옛 값은 계속 노출됩니다** — 그래서 `node_textfile_mtime_seconds` 로 파일이 오래됐는지 함께 감시합니다.

실습 관찰 · 잘못된 파일은 `node_textfile_scrape_error=1` 로 드러났지만 다른 node 메트릭은 계속 수집됐습니다.

복습: exporter를 의심하는 순서

스크레이프가 이상할 때 **무엇부터 열어 볼지**를 정해 둡니다.

증상마다 먼저 볼 메트릭이 다릅니다.

증상	먼저 볼 것
스크레이프가 느림 / timeout	<code>collector_duration_seconds</code> 최댓값
특정 메트릭이 없음	그 collector가 켜져 있나, <code>success</code>
배치 작업 기록이 안 보임	<code>textfile</code> 디렉토리와 <code>mtime</code>
메모리가 부족해 보임	<code>MemFree</code> 를 보고 있진 않은가

전체 스크레이프 시간이 3초입니다. collector가 45개일 때 평균 소요는?

정답: 평균을 구할 수 없습니다. 병렬이라 3초는 **가장 느린 하나**의 시간입니다.

STEP 2 / 4

저장 — 그 값이 어떤 모양으로 쌓이는가

시계열로 저장한다 → 질의한다

메트릭·라벨·샘플

WAL · head · block

인덱스 · 컴팩션

PromQL

라벨 전체가 시계열 하나를 정합니다

시계열 하나를 정하는 것은 이름이 아니라 **라벨 전체**입니다.

라벨은 검색 조건이면서 동시에 비용입니다.

메트릭

무엇을 재는가

`requests_total`

시계열

이름 + 라벨 전체

`{job="api",method="GET"}`

샘플

특정 시각의 값

`12:00 → 842`

`http_requests_total{method="GET",status="200"}`

좋은 라벨

값 종류가 제한적이고 실제로 질문에 씁니다. `method`, `status`, `job`

위험한 라벨

값이 계속 새로 생깁니다.

`user_id`, `request_id`, `trace_id`

사용자 100만 명이면 **시계열 100만 개** — 카디널리티 비용입니다.

블록 하나가 홀로 서는 작은 데이터베이스입니다

네 조각이 **한 디렉토리에** 모여, 다른 블록에 기대지 않고 독립합니다.

추상이 아니라 **눈에 보이는 파일**입니다. 디렉토리 이름은 ULID라 **사전순이 곧 시간순**입니다.

```
/prometheus/data/
├── 01KYPGY9S4SSXES2Y1WVAW82V0/
│   ├── index          199,817 B
│   ├── chunks/000001  333,513 B
│   ├── meta.json      697 B
│   └── tombstones      9 B
├── 01KYQ8M2T1F9RA7X0BNCDE4H3K/
│   └── ... ← 블록마다 같은 네 조각
└── wal/
```

조각	무엇이 들었나
index	라벨에서 chunk 위치를 찾는 역색인
chunks/	압축된 실제 샘플
meta.json	시간 범위·샘플 수 통계
tombstones	지울 구간 표시 지운 게 없어도 파일은 있음

블록끼리 서로를 **가리키지 않습니다**. 그래서 보존 만료가 디렉토리째 삭제로 끝납니다.

타입은 관측 하나가 시계열 몇 개가 되는지를 정합니다

타입은 저장 방식을 바꾸지 않지만 어떤 PromQL 함수를 쓸 수 있는지를 정합니다.

`/metrics`에는 타입이 주석으로 함께 실려 나옵니다. 값만 오는 게 아닙니다.

```
# HELP http_requests_total 총 요청 수
# TYPE http_requests_total counter
http_requests_total{method="GET"} 1027

# HELP node_memory_MemFree_bytes 여유 메모리
# TYPE node_memory_MemFree_bytes gauge
node_memory_MemFree_bytes 2.03e+09
```

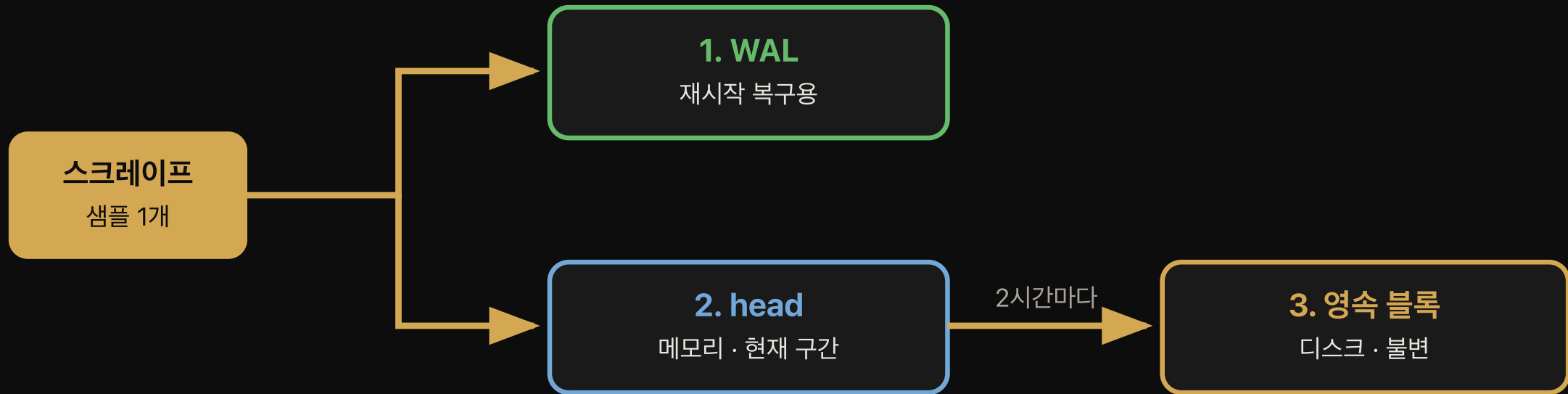
타입	관측 1개 → 시계열	값의 움직임
Counter	1개	오르기만
Gauge	1개	오르내림
Histogram	버킷 수 + 2	버킷마다 카운터
Summary	분위수 개수	관측마다 재계산

TSDB는 타입을 저장하지 않습니다. 어떤 타입이든 디스크에는 (시각, 실수) 쌍으로 똑같이 쌓입니다. 타입은 읽는 쪽 약속이라, `rate()` 를 게이지에 써도 막아 주지 않습니다.

샘플 하나는 WAL과 head로 동시에 갑니다

샘플 하나는 WAL과 head 두 곳으로 동시에 갑니다.

WAL은 내구성을, head는 조회 속도를 말합니다.



▶ 여기서 시연 · 02-01. 스크레이프와 저장 흐름.html 을 열어 단계를 넘겨 가며 봅니다.

WAL은 아직 블록이 아닌 데이터를 지킵니다

head는 메모리에 있습니다. 재시작하면 사라지는데도 데이터가 남는 이유가 WAL입니다.

복구는 디스크에 남은 것부터 차례로 되살리는 세 단계입니다.

재시작 직후 — 메모리의 head 가 통째로 사라진 상태

디스크에는 WAL 과 이미 flush 된 chunk 가 남아 있습니다. 이것만으로 되돌립니다.

① 디스크의 chunk 를 먼저 붙입니다

이미 flush 돼 파일로 있는 것들
읽을 필요 없이 위치만 연결 (mmap)

② 체크포인트를 재생합니다

오래된 WAL 을 미리 요약해 둔 것
쓸모없어진 기록은 이미 걸러져 있음

③ 그 뒤 WAL 만 재생합니다

체크포인트 이후에 들어온 샘플
여기까지 하면 head 가 원상복구

체크포인트가 없으면 어떻게 되나

WAL 을 처음부터 전부 다시 읽어야 합니다. 저자는 예전 버전에서 재생이 30분을 넘는 것을 드물지 않게 봤고,
최적화 뒤 같은 서버가 5분 안팎에 끝난다고 적습니다. 그동안 이 Prometheus 는 질의를 받지 못합니다.

인덱스는 라벨에서 샘플까지 세 단을 거칩니다

질의에 적힌 **라벨**과 **시간 범위**가 각 단계에서 쓰입니다.

사용자가 던진 질의

```
node_cpu_seconds_total{job="kubelet", env="prod"}[12:00 ~ 14:00]
```

← 라벨은 ①로, 시간 범위는 ②로 갑니다

① 포스팅 — 질의의 라벨로 시리즈 ID 를 좁힙니다

job="kubelet" → [10, 11, 17]
 env="prod" → [7, 9, 11, 13, 17]
교집합 → [11, 17]

② ID 11 의 항목 — 이 블록 안 chunk 들의 목록

블록 하나에 이 시계열의 chunk 가 여러 개 있고, 각각 **담당 시간대**가 다릅니다

10:00~12:00
건너뛴

12:00~14:00
질의와 겹침 → 읽음

14:00~16:00
건너뛴

③ 그 chunk 를 가리키는 값이 8바이트짜리 정수 하나입니다

— 이 값은 **인덱스에 이미 적혀 있습니다** (찾는 게 아닙니다)

앞 4바이트 — 몇 번 파일

뒤 4바이트 — 파일 안 위치

블록의 chunks/000001 을 열고 970 바이트로 점프
 거기서 길이만큼 잘라내면 그 chunk 가 나옵니다

chunks/ 안에는 파일이 여러 개입니다 — 세그먼트가 커지면 **000001**, **000002** ... 로 늘어납니다. 그래서 "몇 번 파일"이 따로 필요합니다. 인덱스와 chunk 파일은 **같이 만들어져**, 쓰는 시점에 위치를 적어 둡니다. 이 주소는 **그 블록 안에서만** 유효합니다.

출처: 백전: 03-03 92 인덱스 역색인이 질의를 아키텍처

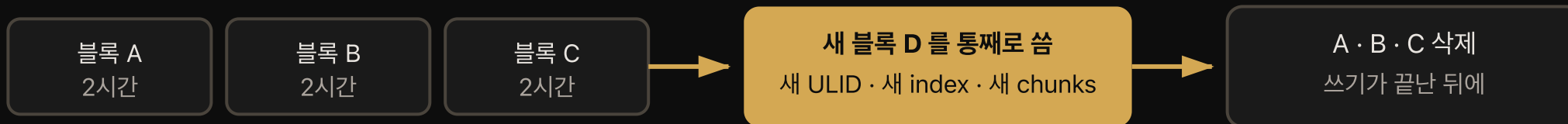
인덱스

컴팩션은 합치는 게 아니라 새로 쓰는 것입니다

작은 블록을 **고쳐서 붙이지 않습니다**. 읽어서 새 블록을 만들고, 원본을 지웁니다.

그렇게 잘 합쳐 두어도 보존 기간이 지나면 폴더째 지웁니다. 왜 지워야 할까요.

컴팩션 — 원본은 읽기만 하고, 새로 씁니다



기존 파일을 고치면 오프셋이 어긋나 읽는 중인 질의가 깨집니다. 새로 쓰고 마지막에 갈아 끼우면 그 위험이 없습니다.

그런데 컴팩션을 해도 디스크는 계속 늘어납니다

컴팩션이 줄이는 것 : 중복된 인덱스

2시간 블록마다 같은 라벨 목록이 반복됩니다.

줄이지 못하는 것 : 샘플 그 자체

15초마다 찍힌 값은 합쳐도 그대로 남습니다.

그래서 안 지우면 디스크가 찹니다

Prometheus 는 서버 한 대의 로컬 디스크에 씁니다. 무한하지 않습니다.

게다가 컴팩션은 **원본과 새 블록을 잠시 같이 들고 있습니다**.

공식 문서가 보존 크기를 가용 디스크의 80~85% 이하로 권하는 이유입니다.

그리고 **한 달 전 15초 단위 값을 볼 일은 드뭅니다**. 오래된 데이터는 가치가 빨리 떨어집니다.

그래도 남겨야 한다면 **지우기 전에 밖으로 내보냅니다** — Thanos 가 새 블록을 S3 로 올리고, 질의 때 로컬과 합쳐 보여 줍니다.

STEP 3 / 4

조회 — 쌓인 값을 어떻게 읽는가

질의한다 → 그래프가 된다

순간 · 범위 벡터

rate

집계 by / without

staleness

실전 질의

대괄호 하나가 순간과 범위를 가릅니다

PromQL은 시계열 묶음을 다루는 언어입니다. 값 하나가 아니라 여러 시계열이 한꺼번에 나옵니다.

함수마다 어느 쪽을 받는지가 정해져 있어, 이 구분이 문법 오류의 절반입니다.

순간 벡터 `up`

시계열마다 지금 값 하나

```
up{instance="a"} 1
```

```
up{instance="b"} 0
```

범위 벡터 `up[5m]`

시계열마다 지난 5분의 값들

그래프로 바로 못 그립니다. 함수로 하나의 값으로 접어야 합니다.

`rate()` · `increase()` 같은 함수는 범위 벡터만 받습니다. `rate(up)` 은 오류이고 `rate(up[5m])` 이 맞습니다.

`rate()` 는 누적값을 속도로 바꿉니다

카운터는 계속 커지기만 해서 **값 자체는 의미가 없습니다**. 초당 얼마나 늘었는지가 궁금한 것입니다.

같은 뼈대에서 **시간으로 나누느냐**만 다른 함수들이 갈라져 나옵니다.

<code>rate</code>	<code>= (마지막 - 처음) ÷ 경과 시간</code>	← 초당 증가량
<code>increase</code>	<code>= 마지막 - 처음</code>	← 기간 내 증가분
<code>irate</code>	<code>= (마지막 - 직전) ÷ 그 간격</code>	← 최근 두 점만

보통은 `rate`

`irate` 는 최근 두 점만 보므로 범위를 늘려도 결과가 거의 같습니다. **빠르게 출력** 이는 **시계열**에만 씁니다.

게이지에 쓰면 틀립니다

`rate · increase` 는 **값이 늘기만 한다고** 전제합니다. 메모리처럼 오르내리는 값에 쓰면 엉뚱한 수가 나옵니다.

집계는 시계열을 접어 개수를 줄입니다

인스턴스마다 나오는 수십 개 시계열을 보고 싶은 단위로 합칩니다.

계산은 `sum` · `avg` 가 합니다. `by` · `without` 은 어느 라벨을 남길지만 정합니다.

```
sum by (service) (rate(http_requests_total[5m]))
```

집계 전 4개 → {service="order-api"} 2.76
 {service="payment-api"} 1.49

by — 남길 것을 적음

닫힌 목록입니다. 새 라벨이 생겨도 결과가 그대로라 변화에 강합니다.

without — 버릴 것을 적음

열린 목록입니다. 새 라벨이 생기면 결과가 쪼개집니다.

그래프의 구멍은 대개 staleness입니다

질의는 5분(`lookback delta`)을 거슬러 값을 찾고, 못 찾으면 시계열이 사라진 것처럼 보입니다.

staleness: 마지막 샘플이 너무 오래돼 더는 유효하지 않다고 보는 상태입니다. 사라져 보이는 경우는 셋입니다.

① 5분간 샘플 없음

lookback delta 기본값을 넘겼습니다. 스크레이프 주기가 5분보다 길면 정상인데도 구멍이 납니다.

② 직전에 있던 게 없어짐

이번 스크레이프에 빠지면 **staleness** 마커가 삽입됩니다. 조건부 등록이 여기 해당합니다.

③ 대상이 사라짐

파드가 삭제돼 감시 목록에서 빠졌습니다. 마지막 값 시각이 빠진 시점과 일치 맞습니다.

화면	실제 상황	해야 할 일
선 없음	그 사유의 실패가 0건	아무것도 안 해도 됨
선 없음	앱이 죽어 메트릭을 못 냄	지금 대응

값이 0이어도 계속 내보내야 둘이 갈립니다 — 0인 평평한 선이 "살아 있다"는 신호입니다.

배운 것을 붙이면 실제 질의가 됩니다

앞에서 본 **Node Exporter**의 **CPU 값**을 실제로 읽어 봅니다.

CPU는 백분율이 아니라 **각 모드에서 보낸 초**로 나옵니다.

```
sum without (cpu, mode) (  
  rate(node_cpu_seconds_total{mode!="idle"}[5m])  
)
```

범위 벡터

[5m] 으로 지난 5분을 씩입니다.

rate

누적 초를 **초당 사용량**으로 바꿉니다.

집계

cpu · mode 를 접어 서버 단위로 합칩니다.

결과는 CPU가 하나면 최대 **1.0**, 둘이면 **2.0** 입니다. 0~100%로 보려면 **코어 수로 나눕니다** — `count without (cpu, mode)`
`(node_cpu_seconds_total{mode="idle"})`

STEP 4 / 4

알림 — 조건이 참일 때 누구에게 어떻게 닿는가

규칙을 평가한다 → 묶어서 보낸다

inactive · pending · firing

그룹핑 · 라우팅

억제 · 시간 구간

룰 테스트 · HA

알림은 두 프로그램이 나눠 맡습니다

알림은 두 프로그램이 나눠 맡습니다. Prometheus는 알림을 보내지 않습니다.

이 구간의 단 하나의 메시지입니다 — 두 상태는 다른 것입니다.

Prometheus

- PromQL 규칙 평가
- inactive / pending / firing
- firing 알림을 전달

Alertmanager

- 그룹핑과 라우팅
- 억제와 침묵
- Slack·메일 발송

▶ 여기서 시연 · 02-02.알림 흐름.html 에서 for 세 시나리오와 그룹핑·억제를 눈으로 비교합니다.

이 장의 단 하나의 메시지 — 두 상태는 다른 것입니다. Prometheus는 알림을 보내지 않습니다.

실습 관찰 · Alertmanager를 멈춰도 Prometheus의 firing은 유지됐고 새 통지만 끊겼습니다.

for 는 순간 이상과 지속 이상을 가릅니다

조건이 참인 **순간**과 **지속**은 다릅니다. 그 둘을 가르는 것이 **for** 입니다.

for 를 채우기 전까지는 **pending**이라 Alertmanager는 아직 아무것도 받지 않습니다.

Prometheus 안에서 — 규칙 평가



조건이 거짓이 되면 pending 은 **0** 으로 **초기화**됩니다. Alertmanager 는 이 구간을 아예 보지 못합니다.

그러면 for 를 얼마로 둘까요

warning — 15분보다 짧지 않게

1~2시간짜리도 겁내지 마십시오.
길수록 조용해집니다.

critical — 15분보다 길지 않게

짧은 값(0m · 5m)은 여기에 씁니다.
빨리 알아야 하는 쪽입니다.

반대편 손잡이 — keep_firing_for

조건이 거짓이 된 뒤에도 유지합니다.
해소·발화가 번갈아 쏟아지는 것을 막습니다.

주의 — 스크레이프가 한 번 실패하면 pending 이 0 으로 돌아갑니다

데이터 공백이 조건을 **거짓처럼** 만듭니다. `last_over_time()` 으로 짧은 공백을 견뎌줍니다.

for 는 룰 평가 주기보다 길어야 합니다 — 실습에서도 **for: 10s** 는 30초 주기에서 바로 firing 됐고, **for: 2m** 에서만 pending 이 보였습니다.

그룹핑은 폭풍을 한 묶음으로 바꿉니다

알림 100건을 한 통으로 묶는 것이 그룹핑입니다.

노드 100대가 한꺼번에 죽으면 메일 100통이 아니라 한 통이어야 합니다.

```
route:
  group_by: [alertname, cluster]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
```

설정

언제 쓰이나

`group_by`

이 라벨이 같으면 한 묶음

`group_wait`

첫 알림 뒤 모으는 시간

`group_interval`

그룹에 변경이 생겼을 때

`repeat_interval`

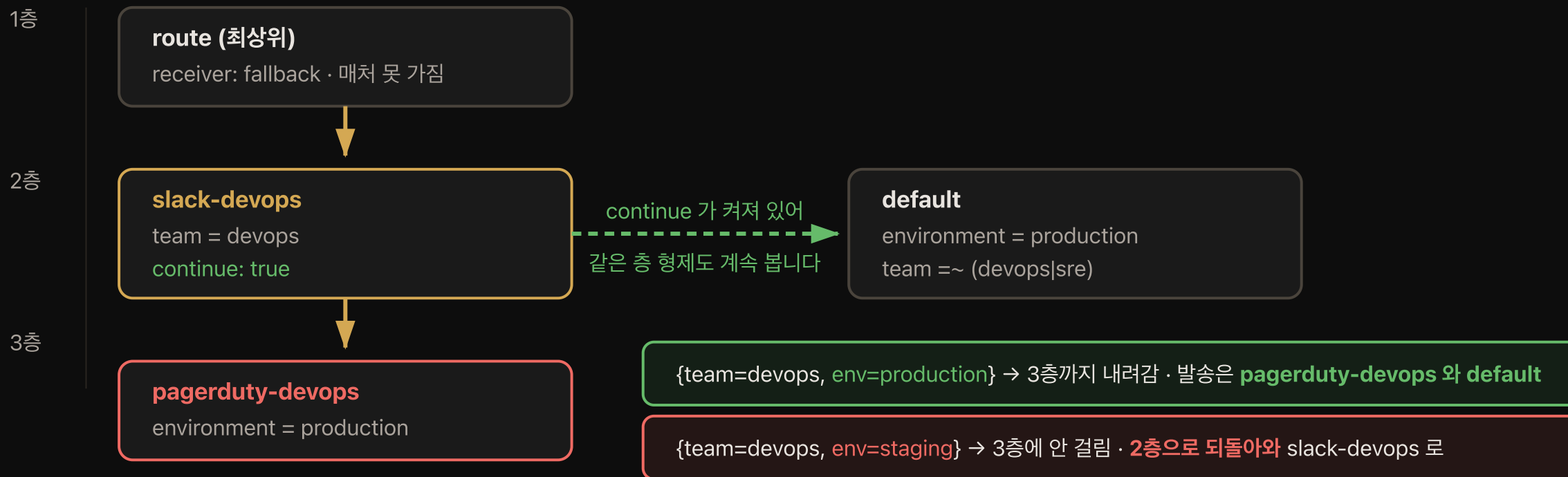
새 발화도 해소도 없을 때 다시

"변화 없어도"가 뜻하는 것 : 마지막 `group_interval` 이후 새로 발화하거나 해소된 알림이 하나도 없는 상태입니다. 값이 그대로 다시 들어오는 것은 변화가 아닙니다. 하나라도 바뀌면 반복 대신 갱신이 나옵니다.

라우트 트리는 층층이 내려가며 수신자를 정합니다

알림 하나가 트리를 걸어 내려가며 **가장 구체적인 라우트**를 찾습니다.

자식에 걸리면 더 내려가고, 어느 자식에도 안 걸리면 부모로 되돌아옵니다.



첫째 줄에서 **slack-devops** 는 지나가기만 합니다 : 자식에 걸려 더 내려갔으니 경유지입니다. 지나간 노드가 곧 발송지는 아닙니다.

`mute_time_intervals` 는 그 시간대에 발송만 멈추고 탐색은 그대로입니다. **최상위에는 못 걸어** 바로 아래에 매처 없는 라우트를 두고 겁니다.

억제는 원인이 결과 알림을 가립니다

원인 알림이 살아 있는 동안 결과 알림의 통지를 막습니다.

DB가 죽으면 API 오류도 함께 터집니다. 당직자가 볼 것은 원인 하나입니다.

```
inhibit_rules:  
  - source_matchers:  
      [alertname="DBDown"]  
    target_matchers:  
      [alertname="APIHighError"]  
    equal: [cluster, service]
```

필드

뜻

source_matchers

가리는 쪽 — 원인

target_matchers

가려지는 쪽 — 결과

equal

이 라벨 값이 같을 때만

equal 이 없으면 다른 클러스터의 장애까지 가려집니다.

가려진 알림도 Prometheus에서는 여전히 firing입니다. 통지만 막힐 뿐입니다.

`promtool test rules`로 알림을 테스트합니다

알림도 코드입니다. 기대대로 발화하는지 테스트할 수 있습니다.

알림은 Prometheus 안에서 정의·평가되므로 `amtool`이 아니라 `promtool`을 씁니다.

<code>promtool check rules rule.yaml</code>	문법만 검사
<code>promtool test rules unit.yaml</code>	기대대로 발화하는지 검사

왜 `amtool`이 아닌가

알림은 Prometheus 안에서 정의되고 평가되기 때문입니다. `amtool`은 Alertmanager 쪽 검증입니다.

무엇을 단언하나

시계열을 지어 넣고 "몇 분에 무엇이 어떤 라벨로 발화해야 하는지"를 적습니다.

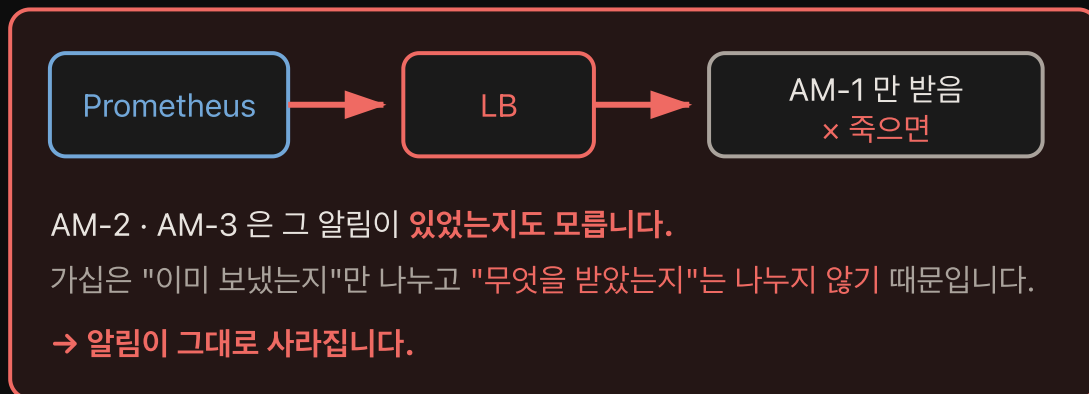
사고 시간대에 쿼리를 돌려 보는 방식은 유효하지만, 룰을 손본 뒤에도 계속 동작하리라는 보장은 주지 못합니다. 알림도 코드입니다.

Alertmanager 앞에 로드밸런서를 두지 마십시오

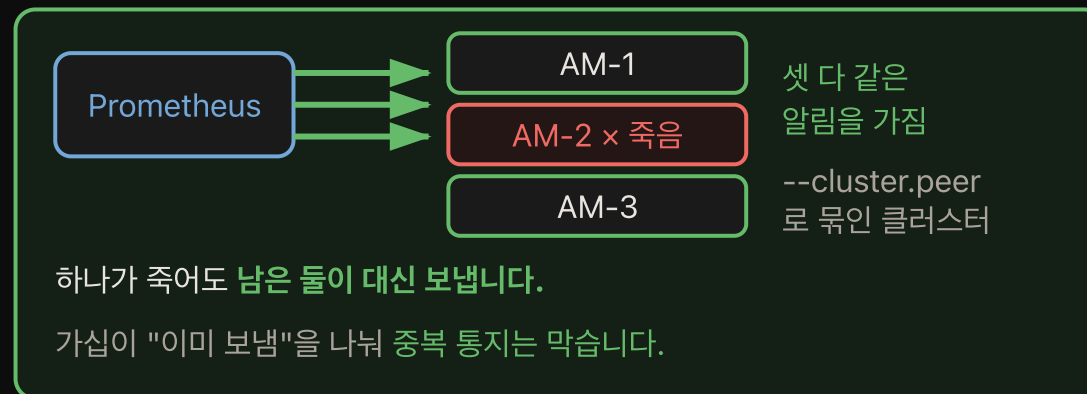
가십은 **일부만** 공유합니다. 그래서 앞에 로드밸런서를 두면 **알림이 유실됩니다**.

Prometheus가 모든 인스턴스에 같은 알림을 직접 보내야 합니다.

× 로드밸런서를 두면



○ 모든 인스턴스에 직접



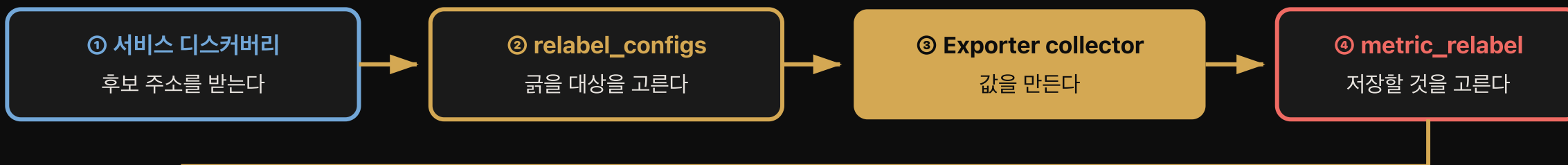
인스턴스끼리는 `--cluster.peer` 로 서로를 피어로 지정해 클러스터를 이룹니다. 합의 알고리즘을 쓰지 않아 정족수가 필요 없고, 한 대만 살아 있어도 알림은 계속 나갑니다. 보통 2~3대를 서로 다른 AZ에 둡니다.

한 번의 스크레이프가 끝까지 가는 길

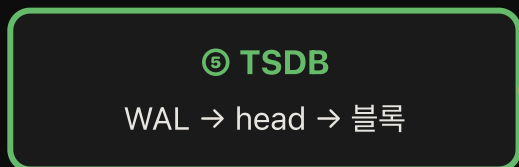
오늘 본 네 구간이 각각 이 흐름의 한 토막이었습니다.

맨 앞에서 본 다섯 동사가 여기서 여섯 단계로 완성됩니다.

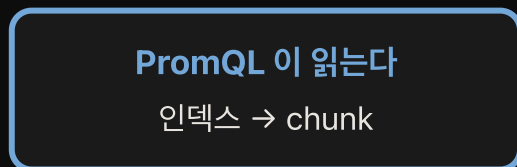
수집



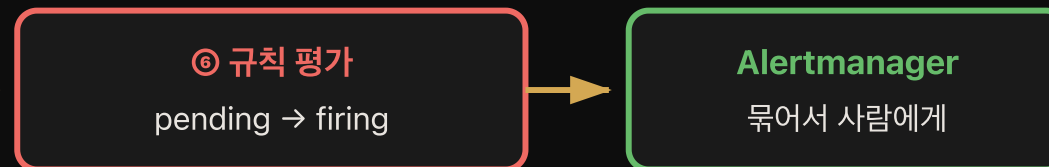
저장



조회



알림



같은 문법(relabeling)이 ②와 ④ 두 자리에 놓입니다. 거기서 거르는 것이 각각 타겟과 시계열로 갈립니다.

어느 칸에서 값이 사라졌는지가 곧 어디를 고쳐야 하는지입니다.

마지막 5문장 회상

보지 않고 설명할 수 있다면 네 주제의 뼈대를 잡은 것입니다.

다섯 문장으로 압축했습니다.

1. 라벨 전체가 시계열 하나를 정합니다.
2. 샘플은 WAL → head → block으로 이동하고, 블록은 불변입니다.
3. relabeling은 타깃과 시계열을 서로 다른 위치에서 거릅니다.
4. Prometheus의 firing과 Alertmanager의 억제 는 다른 상태입니다.
5. Node Exporter timeout은 가장 느린 collector의 duration부터 봅니다.

보지 않고 설명할 수 있다면 네 장의 뼈대를 잡은 것입니다.

다음은 한 대를 넘어서는 이야기입니다 — 06장 샤딩·HA · 07장 카디널리티 디버깅 · 09장 remote write · 10장 Thanos.